

HelixCode

HelixCode

Дистрибуирана AI развојна платформа која дели посао, чува га и никада не губи ваше место.

Сажетак

HelixCode је предузетничка, Go-базирана дистрибуирана AI развојна платформа која дели развојне задатке на интелигентно подељене подзадатке распоређене по мрежи радника управљаних путем SSH, са аутоматским чувањем стања и повратком на претходне верзије како ни један део посла никада не би био изгубљен. Она обједињује интеграцију више LLM провајдера, радне токове током целог развојног циклуса и испоруку на више платформи иза REST, CLI, TUI и MCP интерфејса.

Кратак опис

HelixCode је дистрибуирана AI развојна платформа написана у Go. Дели посао на интелигентне задатке распоређене по мрежи радника заснованој на SSH, чува напредак аутоматским чувањем стања и повратком на претходне верзије, интегрише више LLM провајдера и покреће цео развојни циклус преко REST, CLI, TUI и MCP интерфејса.

Детаљан опис

HelixCode је предузетничка дистрибуирана AI развојна платформа (`dev.helix.code`, MIT) изграђена око једноставног обећања које њен слоган дословно испуњава: подели посао, сачувај га и никада не изгуби своје место. Дизајнирана је за интелигентну поделу задатака, аутоматско чување напретка и развојне токове на више платформи, а написана је у Go због конкурентности и преносивости у виду једног бинарног фајла

које дистрибуирано рачунарство захтева — са аутоматским чувањем стања, повратком на претходне верзије и праћењем у реалном времену као основним, а не опционим функцијама.

Њена архитектура поставља REST + WebSocket + MCP API слој изнад скупа фокусираних основних сервиса — JWT аутентификацију и управљање сесијама, управљање базеном радника заснованим на SSH са праћењем стања, управљање задацима са чувањем стања и руковањем зависностима, управљање пројектима и радним токовима, као и обједињени слој LLM провајдера — све сачувано на PostgreSQL, док је Redis доступан као опциони слој за координацију и кеширање. Дистрибуирани радници се аутоматски инсталирају преко мреже, тако да је проширивање система једноставно усмеравање сервера на машину уместо ручног подешавања, а мулти-клиент интерфејси обухватају CLI, терминалски интерфејс, REST и мобилне оквири, чиме је иста платформа доступна из скрипте, терминала или апликације.

HelixCode покреће комплетан развојни циклус од почетка до краја: планирање, изградња, тестирање и рефакторисање се извршавају аутоматски уз праћење зависности и контекста више сесија, тако да дугорочни напори задржавају своју нит кроз прекиде и границе између машина. Интегрише више LLM провајдера — лама.цпп, Ollama и OpenAI — иза једног интерфејса, а затим додаје избор модела свестан хардвера који детектује доступне CPU/GPU/меморију и прилагођава модел машини, као и подршку за напредне стратегије резоновања попут ланчаног размишљања и стабла размишљања за проблеме који захтевају више од једног пролаза. Model Context Protocol је имплементиран преко више транспортних протокола за стандардизовану размену алата и контекста, а обавештења на више канала (Slack, Discord, Емаил, Telegram) обавештавају тимове о напретку дистрибуираног рада. Подржане су платформе Linux, macOS, Windows, Aurora OS и SymphonyOS.

Зашто смо га изградили

Развој који се одвија на више машина уз помоћ AI обично губи контекст и напредак када се задаци раздвоје или прекину. HelixCode је створен да поделу задатака учини интелигентном, а очување рада аутоматским — како би се велики развојни напор могао разложити, распоредити по мрежи радника, контролисати тачкама провера и наставити или вратити уназад без губитка стања.

Зашто је револуционаран

Он чини развој уз помоћ AI *издржљивим* — способност која никада раније није била практична када су тимови ручно спајали ове делове. Три ствари које обично постоје у три одвојена алата сада постају једна платформа: дистрибуирано израчунавање (мреже радника SSH са аутоматском инсталацијом и праћењем здравља), помоћ у развоју уз AI (LLM-ови са више пружалаца услуга, способни за резонување и позивање алата) и аутоматизација радних токова током целог животног циклуса. Спојно ткиво је провера тачака ослоњена на базу података: пошто се стање задатка, тачке провера и зависности чувају у PostgreSQL, посао који се протеже преко више машина и сесија може се вратити уназад или наставити тачно тамо где је стао. Прекиди и подела рада престају да буду извор изгубљеног напретка и постају рутински, обновљиви догађаји.

Шта је иновативно

- Очување рада као основни принцип: аутоматска провера тачака и враћање уназад примењено на *дистрибуиране* развојне задатке, тако да напредак преживи прекиде и отказивање машина уместо да нестане с њима.
- Избор модела свесног хардвера који анализира детектовани CPU/GPU/меморију и сваком задатку додељује модел који машина заиста може добро да покрене — без ручног подешавања по радницима.
- Једна платформа, пет улаза: REST, WebSocket, CLI, TUI и MCP, при чему је сам MCP доступан преко више протокола тако да се алати и агенти могу интегрисати без обзира на начин повезивања.
- Подршка за више платформи која превазилази уобичајени трио десктоп система и укључује Aurora OS и SymphonyOS, проширујући мрежу радника на платформе које већина алата игнорише.

Највећи технички изазови и како смо их решили

- **Да се не изгуби рад на дистрибуираним, прекидивим задацима.** Када је посао подељен на више машина, сваки пад система или прекид обично оставља у ваздуху све што је било у току. Ми смо задатак моделовали као носиоца тачака провера и зависности, сачуваних у PostgreSQL, тако да систем може да се врати на последње исправно стање или настави одатле — издржљивост која постоји у слоју података, а не у крхом стању у меморији.

- **Управљање хетерогеном мрежом радника.** Мрежа машина са Linuxом, macOS-ом, Windowsом, Аурором и SymphonyOS-ом стално се мења у погледу доступности и подешавања. Ми то решавамо посвећеном услугом за управљање базеном радника која обавља регистрацију засновану на SSH, аутоматску инсталацију на нове чворове и непрестано праћење здравља, тако да мрежа остаје позната и контролисана како машине долазе и одлазе.
- **Различитост пружалаца услуга и хардвера.** LLM бацкендови и машине на којима се покрећу веома се разликују по могућностима. Ми смо то сакрили иза јединственог интерфејса за пружаоце LLM и упарили га са детекцијом хардвера (CPU/GPU/меморија) која покреће интелигентан избор модела, тако да прави модел стигне на праву машину без потребе да корисник размишља о било чему од тога.

Технолошки стек

- **Go (1.26+ унутрашњи модул)** — изабран јер његова конкурентност заснована на горутинама и излаз у виду једног бинарног фајла управо одговарају потребама дистрибуираног радничког система: јефтин паралелизам за оркестрацију и самостални бинарни фајл који се аутоматски инсталира на било који чвор. Садржи све основне сервисе и бинарне фајлове CLI/сервера.
- **Gin (ХТТП оквир)** — изабран због брзог, минималистичког слоја REST са ниским режијским трошковима; служи /api/v1 површину (аутентификација, радници, задаци, пројекти) са којом комуницира сваки клијент.
- **PostgreSQL 15+ (путем pgx/v5)** — изабран као трајан систем евиденције јер чување стања и повратак на претходну верзију захтевају трансакциону перзистенцију; садржи шему дистрибуираног рачунарства са 11 табела (корисници, радници, задаци, пројекти, сесије, добављачи LLM-а, обавештења) која омогућава очување рада.
- **Redis 7+ (опционо, го-редис/в9)** — изабран као опциони слој кеширања и координације који убрзава „вруће“ путеве без претварања у обавезну зависност, тако да минимална имплементација и даље функционише само са Постгресом.
- **SSH** — изабран као транспорт за контролу радника управо зато што је већ свуда присутан и већ обезбеђен; покреће регистрацију радника, аутоматску инсталацију и извршавање удаљених команди широм целог кластера без потребе за претходним постављањем посебног агента.
- **Model Context Protocol (MCP)** — изабран за стандардизовану размену алата и контекста како би се спољашњи алати и агенти интегрисали преко једног отвореног протокола; имплементиран са подршком за више

транспортних механизма како би се прилагодио клијентима где год се повежу.

- **Добављачи LLM (лама.цпп, Ollama, OpenAI)** — изабрани да обухвате и локално и хостовано извођење закључивања иза јединственог интерфејса, тако да се избор хардвера може усмерити на локални модел или хостовани, а позивалац не мора да зна разлику.

Статус и напомене о искрености

- **Статус: бета.** У РЕАДМЕ фајлу стоји да је пројекат „ПОТПУНО ЗАВРШЕН / свих 5 фаза”; та потпуност је наведена од стране пројекта, а не независно потврђена, па се страница према томе односи као према бети.
- Све наведене појединости потичу из РЕАДМЕ фајла у репозиторијуму; маркетиншке фразе (слогани) су уредничке природе, а не метрички подаци из извора.

Приоритетни ниво: Helix-основни.